Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»
Университетский технологический колледж
Кафедра Информационные системы в экономике

Отчет защищен с оценкой _____
Преподаватель _____С.В.Умбетов
« » _____ 2026 г.

Отчет по лабораторной работе №3 по дисциплине «Устройство и функционирование интеллектуальных информационных систем» «Введение в обработку текста (NLP Basics) на русском языке»

ЛР 09.02.08.23.003

Студент группы 1ИИС-41 М.А. Ситников
группа фи.о., фамилия
Преподаватель старший преподаватель, доцент С.В. Умбетов
должность, ученая степень фи.о., фамилия

Барнаул 2026

Лабораторная работа №3
Введение в обработку текста (NLP Basics) на русском языке

Цель работы: изучить базовые этапы предобработки текста: токенизацию, очистку с помощью регулярных выражений, удаление стопслов, лемматизацию с помощью spaCy и NLTK, а также векторизацию текста методом TF-IDF.

Задачи к работе:

1. Установить и загрузить необходимые библиотеки (spaCy, русскоязычную модель ru_core_news_sm, NLTK со списком стоп-слов).
2. Сформировать исходный корпус текстов на русском языке.
3. Выполнить очистку текста от «шума» с помощью регулярных выражений (RegEx).
4. Провести токенизацию, удаление стоп-слов и лемматизацию с использованием spaCy.
5. Обернуть все этапы предобработки в единый пайплайн и обработать весь корпус.
6. Вручную рассчитать метрику TF-IDF для векторизации документов.
7. Выполнить самостоятельные задания по кастомизации стоп-слов, исследованию масштабируемости корпуса и модификации регулярных выражений. 8. Ответить на контрольные вопросы.

Задание принято:
Ситников М.А.



Подпись

Ход работы

Шаг 1: Установка и загрузка библиотек

Для работы с русским языком нам понадобятся библиотека spaCy и её официальная русскоязычная модель, а также инструменты из библиотеки NLTK.

```
!pip install spacy
!python -m spacy download ru_core_news_sm
```

Рис.1 – Установка и загрузка spaCy и её официальной русскоязычной модели

```
[2]
10
сек.
import re
import math
import spacy
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')
nlp = spacy.load("ru_core_news_sm")
print("\n\nComplete!")

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.


Complete!
```

Рис.2 – Загрузка библиотеки NLTK со списком стоп-слов русского языка

Шаг 2: Исходные данные (Корпус текстов)

Определим простой список (корпус) из четырех фраз на русском языке.

```
[3]
0
сек.
corpus = [
    "Сегодня была отличная погода, и мы пошли гулять в красивый парк!",
    "Ужасная погода... Весь день идёт сильный дождь, прогулка отменяется.",
    "В парке мы видели красивых птиц. Погода шепчет, солнце светит ярко.",
    "Дождь не прекращается! Сильный ветер сорвал все планы на вечер."
]

sample_text = corpus[0]
print(f"Исходный текст для анализа: {sample_text}")

Исходный текст для анализа: Сегодня была отличная погода, и мы пошли гулять в красивый парк!
```

Рис.3 – Создание исходного корпуса документов

Шаг 3: Очистка текста с помощью Регулярных выражений (RegEx)

Первый шаг — удалить знаки препинания, цифры и привести текст к нижнему регистру, оставив только русские буквы и пробелы. Шаблон r’[^а-яё\S]’, ‘’ оставляет все русские символы, в том числе букву «ё» и пробелы

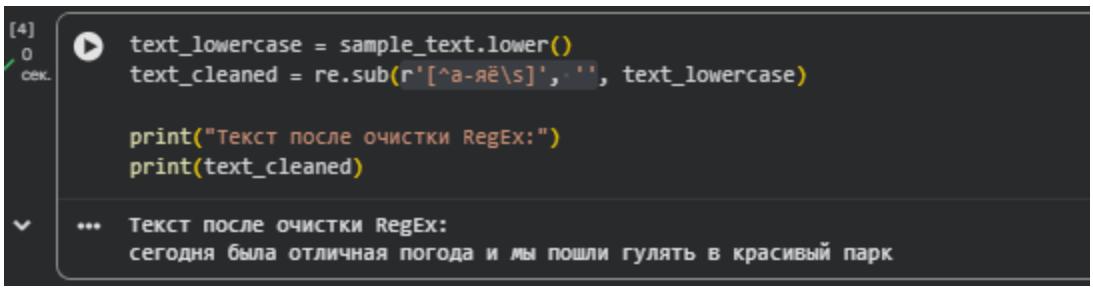


Рис.4 – Результат применения шаблона r’[^а-яё\S]’ в тексте

Шаг 4: Токенизация, удаление стоп-слов и Лемматизация (spaCy)

Используем spaCy для того, чтобы:
1. Разбить текст на отдельные слова (токенизация).
2. Отфильтровать «мусорные» слова (стоп-слова, такие как и, в, мы), используя список из NLTK.
3. Привести слова к начальной форме (лемматизация, например, «пошли»

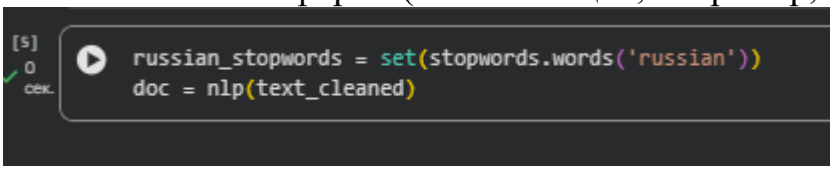


Рис.5 – Получение списка русских стоп-слов из NLKT и разделение очищенного текста text_cleaned на токены с помощью nlp

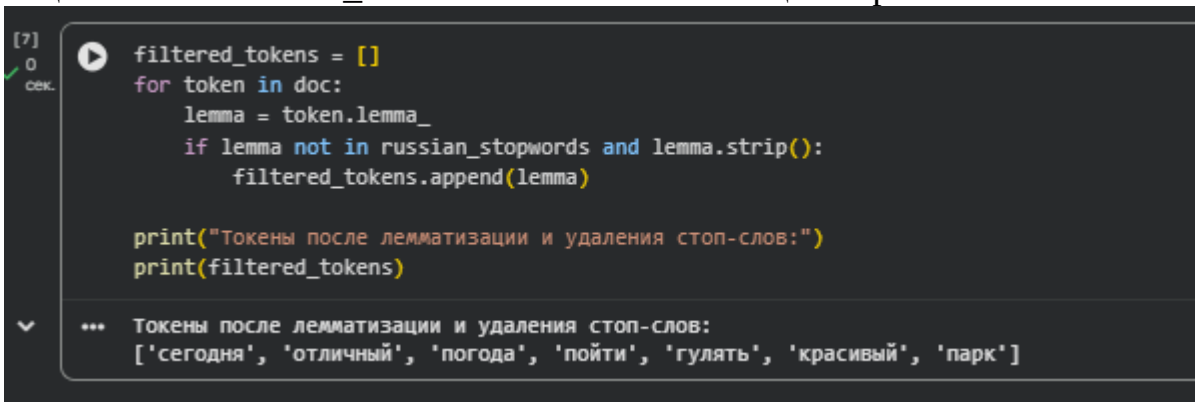


Рис.6 – Приведение слов к начальной форме командой lemma = token.lemma_. Удаление «мусорных» слов из списка русских стоп-слов NLKT(таких как и, в, мы) и лишних пробелов. Добавление результата в filtered_tokens и его вывод.

Шаг 5: Полная предобработка всего корпуса

Обернем предыдущие шаги в одну функцию и обработаем весь наш список документов (corpus).

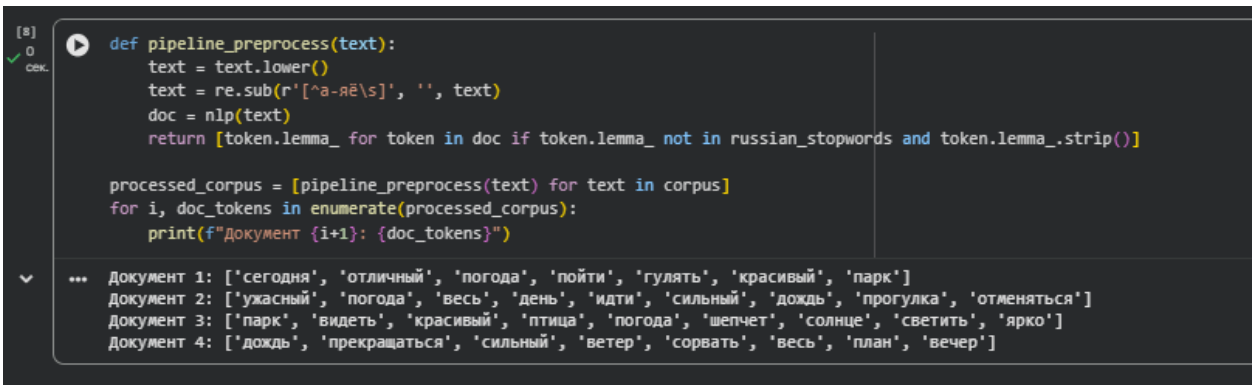


Рис.7 – Все предыдущие шаги обёрнуты в одну функцию pipeline_preprocess и обработка всего списка документов

Шаг 6: Векторизация текста (Расчет TF-IDF вручную)

Для понимания математической сути TF-IDF (Term Frequency — Inverse Document Frequency), полезно рассчитать метрику базовыми формулами, без использования тяжелых библиотек вроде scikit-learn.

1. TF (Частота слова) = (Сколько раз слово встречается в документе) / (Всего слов в документе).
2. IDF (Обратная частота документа) = log (Всего документов / Сколько документов содержат это слово).
3. TF-IDF = TF × IDF

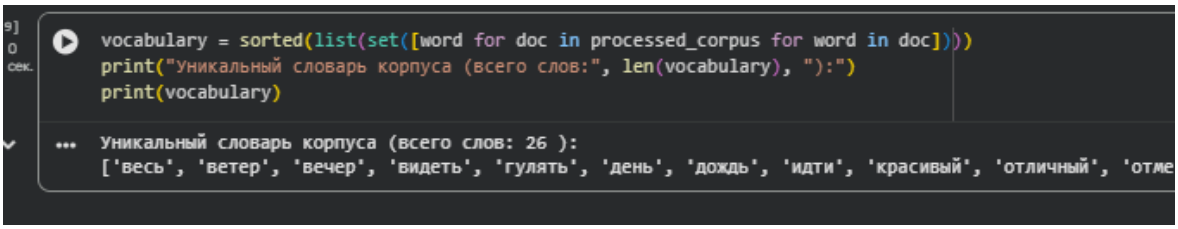


Рис.8 – Создание уникального корпуса

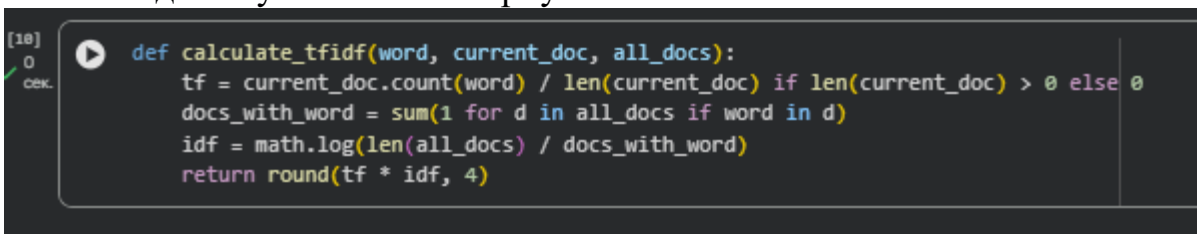


Рис.9 – Расчёт TF(Сколько раз слово встречается в документе / Всего слов в документе) и IDF log (Всего документов / Сколько документов содержат это слово). Расчёт TF-IDF = TF × IDF.

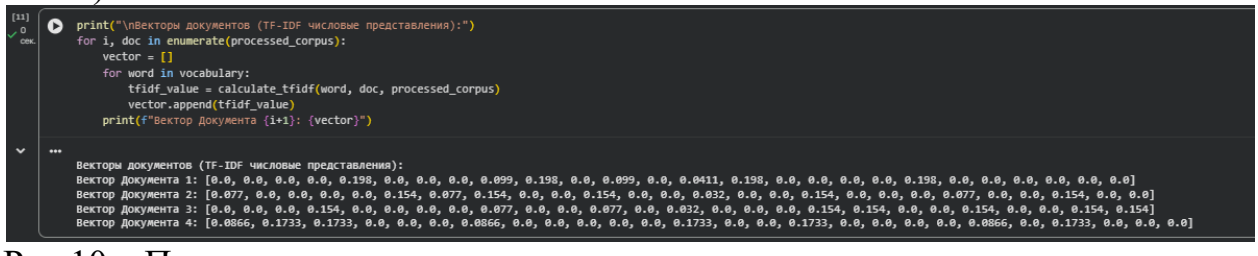


Рис.10 – Построение векторов для каждого документа

Задания для самостоятельной работы студентов

Задание 1. Глубокий анализ и кастомизация стоп-слов

1. Напишите код, который выводит на экран общее количество слов в загруженном списке russian_stopwords, а также сам список (выведите его в алфавитном порядке).

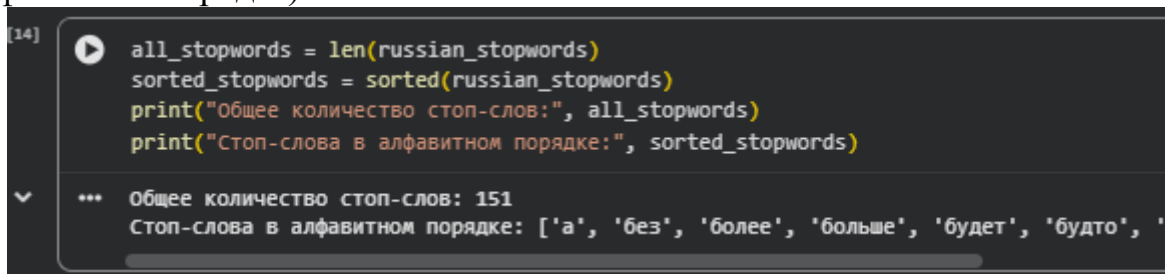


Рис.11 – Выведено общее количество стоп-слов и список этих слов в алфавитном порядке

2. Найдите, выпишите в отчет и классифицируйте 5 любых стоп-слов (например: 1 личное местоимение, 1 предлог, 1 союз, 1 частица, 1 указательное слово).

№	Стоп-слово	Часть речи/ класс
1	а	Союз(противительный)/ частица
2	без	Предлог
3	более	Наречие (сравнительной степени)
4	больше	Наречие (сравнительной степени)
5	будет	Глагол-связка (форма «быть»)

3. Добавьте в список стоп-слов два кастомных контекстно-зависимых слова, которые часто встречаются в нашем корпусе (например, слова «сегодня» и «весь»). Очистите корпус заново и покажите в отчете, как изменился итоговый список токенов для первого документа.

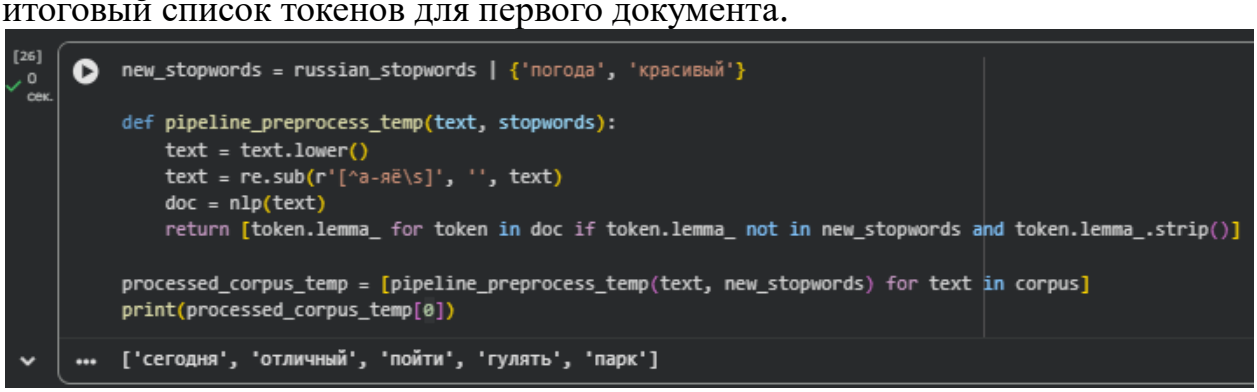


Рис.12 – В список стоп-слов добавлены кастомные слова «погода» и «красивый». Проведена повторная очистка и выведен итоговый список токенов для первого документа.

Результат: В первом документе исчезли слова «погода» и «красивый» (было 7 токенов, стало 5).

Задание 2. Исследование масштабируемости корпуса и динамики векторов

1. Придумайте и добавьте в список corpus пятый документ на русском языке. Обязательное условие: предложение должно содержать как минимум два слова, которых ранее не было в словаре, и два слова, которые уже встречались в прошлых документах.

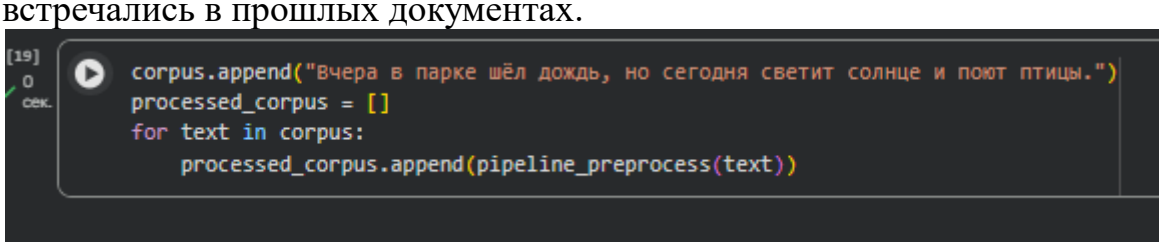


Рис.13 – Добавление нового документа, который содержит 3 новых слова («вчера», «поют» и «шёл»)

2. Перезапустите пайплайн обработки. Сравните и зафиксируйте в отчете: о Каким был размер уникального словаря (vocabulary) до добавления текста, и каким он стал после? о Как изменилась длина итоговых числовых векторов для каждого документа?



Рис.14 – Добавление нового документа к другим документам. Сравнение исходных данных и изменённых

До добавления документа:
Уникальный словарь: 26 слов
Длина вектора каждого документа: 26 элементов
После добавления документа:
Уникальный словарь: 29 слов (добавились: «вчера», «петь» и «шёл»)
Длина вектора каждого документа: 29 элементов

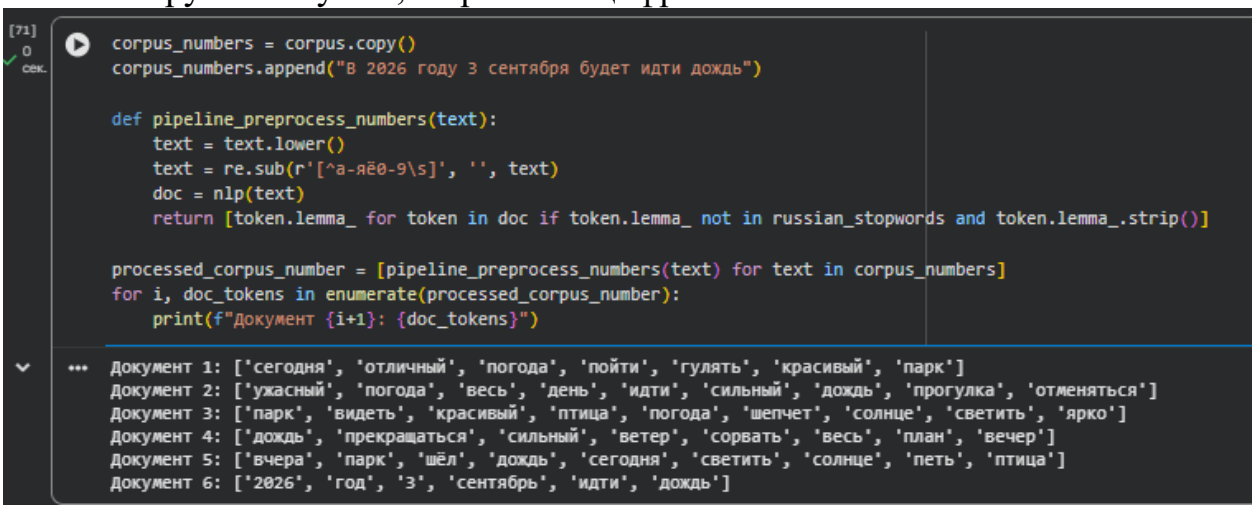
3. Математическая интерпретация: Посмотрите на значения TF-IDF для новых слов в пятом документе. Почему их вес (значение) оказался выше, чем у слов, которые дублировались из прошлых текстов? Обоснуйте ответ, используя формулу IDF.

Расчёт для слова «вчера» (встречается только в документе 5):
TF = 1/9 ≈ 0.111(1 вхождение в документе из 9 токенов)
IDF = ln(5/1) = ln(5) ≈ 1.609(встречается в 1 из 5 документов)
TF-IDF = 0.111 × 1.609 ≈ 0.178
Расчёт для слова «петь» (встречается только в документе 5):
TF = 1/9 ≈ 0.111
IDF = ln(5/1) = ln(5) ≈ 1.609
TF-IDF = 0.111 × 1.609 ≈ 0.178
Расчёт для слова «шёл» (встречается только в документе 5):
TF = 1/9 ≈ 0.111
IDF = ln(5/1) = ln(5) ≈ 1.609
TF-IDF = 0.111 × 1.609 ≈ 0.178
Расчёт для слова «парк» (встречается в 3 документах: 1, 3, 5):
TF = 1/9 ≈ 0.111
IDF = ln(5/3) = ln(1.667) ≈ 0.511
TF-IDF = 0.111 × 0.511 ≈ 0.057
Расчёт для слова «дождь» (встречается в 3 документах: 2, 4, 5):
TF = 1/9 ≈ 0.111
IDF = ln(5/3) = ln(1.667) ≈ 0.511
TF-IDF = 0.111 × 0.511 ≈ 0.057
Расчёт для слова «солнце» (встречается в 2 документах: 3, 5):
TF = 1/9 ≈ 0.111
IDF = ln(5/2) = ln(2.5) ≈ 0.916
TF-IDF = 0.111 × 0.916 ≈ 0.102

Результат: Вес новых уникальных слов («вчера», «петь» и «шёл») оказался выше, чем у слов, дублировавшихся из прошлых текстов («парк», «дождь» и «солнце»), потому что компонент IDF обратно пропорционален частоте встречаемости слова в корпусе. Чем реже слово встречается в документах, тем выше его IDF и TF-IDF. Слово, которое встречается во всех документах (например, если бы «погода» осталась в 5-м документе и не была удалена как кастомное стоп-слово), получило бы IDF = ln(5/5) = 0 и полностью обесценилось бы - оно не несёт дифференцирующей информации. Это и есть смысл метрики TF-IDF: она повышает вес редких, специфичных терминов и понижает вес общеупотребительных.

Задание 3. Модификация регулярных выражений под сохранение числовых данных

1. Добавьте в один из документов корпуса точную временную или количественную метрику (например, «в 2026 году» или «3 плана»). Измените регулярное выражение в функции pipeline_preprocess. Текущий шаблон r'[^a-яё\s]' полностью стирает цифры. Модифицируйте его так, чтобы он сохранял в тексте и русские буквы, и арабские цифры.



```
[71]
0
сек.
corpus_numbers = corpus.copy()
corpus_numbers.append("В 2026 году 3 сентября будет идти дождь")

def pipeline_preprocess_numbers(text):
    text = text.lower()
    text = re.sub(r'[^a-яё0-9\s]', '', text)
    doc = nlp(text)
    return [token.lemma_ for token in doc if token.lemma_ not in russian_stopwords and token.lemma_.strip()]

processed_corpus_number = [pipeline_preprocess_numbers(text) for text in corpus_numbers]
for i, doc_tokens in enumerate(processed_corpus_number):
    print(f"Документ {i+1}: {doc_tokens}")

Документ 1: ['сегодня', 'отличный', 'погода', 'пойти', 'гулять', 'красивый', 'парк']
Документ 2: ['ужасный', 'погода', 'весь', 'день', 'идти', 'сильный', 'дождь', 'прогулка', 'отменяться']
Документ 3: ['парк', 'видеть', 'красивый', 'птица', 'погода', 'шепчет', 'солнце', 'светить', 'ярко']
Документ 4: ['дождь', 'прекращаться', 'сильный', 'ветер', 'сорвать', 'весь', 'план', 'вечер']
Документ 5: ['вчера', 'парк', 'шёл', 'дождь', 'сегодня', 'светить', 'солнце', 'петь', 'птица']
Документ 6: ['2026', 'год', '3', 'сентябрь', 'идти', 'дождь']
```

Рис.15 – Добавлен новый документ с числовыми значениями. С помощью шаблона r'[^a-яё0-9\s]', теперь сохраняются не только русские буквы, но и арабские цифры

2. Прогоните обновленный текст через модель spaCy. Посмотрите, как токенизатор обработал комбинацию цифр и букв. Стала ли строка "2026" отдельным токеном? Какой начальный вид (лемму) присвоила система числительному?

Строка «2026» и «3» стали отдельными токенами — токенизатор spaCy корректно выделяет последовательности цифр как самостоятельные единицы. Лемма, присвоенная числительным: «2026» → «2026», «3» → «3». Для числительных лемма совпадает с исходной формой, так как у чисел нет флективных изменений в том же смысле, что у существительных или глаголов.

3. Подумайте и напишите в выводе, в каких реальных задачах NLP сохранение цифр критически важно (например, анализ чеков, парсинг адресов), а в каких — это избыточный «шум», который нужно удалять.

Задачи, где цифры критически выажны:
Анализ финансовых документов и чеко — суммы, номера счетов, даты платежей несут ключевую смысловую нагрузку.
Парсинг адресов и геоданных — номера домов, почтовые индексы, координаты.
Медицинские записи — дозировки, даты приёмов, показатели анализов.
Техническая документация — версии ПО, артикулы, серийные номера.
Анализ новостей и отчётов — годы, статистические показатели, проценты.

Задачи, где цифры — избыточный «шум»:
Тематическое моделирование художественной литературы — числа редко несут тематическую нагрузку.
Анализ тональности отзывов — конкретные цифры (например, «купил 2 штуки») обычно не влияют на эмоциональную окраску.
Кластеризация новостных статей по темам — годы и числа могут создавать ложные кластеры.
Поиск семантической близости документов — числа увеличивают размерность пространства признаков без добавления смысла.

Ссылка на GitHub:
https://github.com/Enot2130/Sitnikov_1iis41/tree/main/Устройство%20и%20функционирование%20интеллектуальных%20информационных%20систем/Введение%20в%20обработку%20текста%20(NLP%20Basics)%20на%20русском%20языке

Ссылка на Google Colab:
https://colab.research.google.com/drive/114ufyaYoWEfDb_L8NnzhvdOl6WpkOi7T?usp=sharing

Контрольные вопросы

1. Какова роль регулярных выражений (RegEx) при предварительной очистке текстового шума?

Они выполняют фильтрацию «шума»: удаляют веб-теги, системные символы, знаки препинания и числовые значения. Применение регулярных выражений, основанных на теории конечных автоматов и формальных языков, позволяет нормализовать регистр и устранить избыточные пробельные символы, чтобы подготовить текст к дальнейшему разбиению.

2. К каким ошибкам в анализе может привести избыточное удаление спецсимволов и дефисов (например, в словах «e-mail» или «он-лайн»)?

Ошибки на этапе токенизации, такие как неверная обработка дефисов или сокращений, ведут к искажению всего последующего анализа. Избыточное удаление может разрушить структуру составных слов.

3. Почему приведение текста к нижнему регистру (lowercase) является обязательным шагом перед токенизацией?

Нормализация регистра является частью этапа очистки текста с использованием RegEx. Это необходимо для того, чтобы подготовить текст к дальнейшему разбиению (токенизации) и избежать восприятия одинаковых слов, написанных с заглавной и строчной буквы, как разных сущностей.

4. Как настроить регулярное выражение, чтобы оно сохраняло в тексте и буквы, и важные цифровые данные (например, даты или артикулы)?

Нужно добавить диапазон цифр в исключающий класс символов: r'[^a-яё0-9\s]'. Этот шаблон оставит русские буквы (включая «ё»), арабские цифры и пробельные символы, удалив всё остальное. Для сохранения дат в формате «01.01.2026» можно расширить шаблон до r'[^a-яё0-9.\s]'.

5. В каких задачах обработки текста нельзя удалять знаки препинания и смайлики (emoji)?

В анализе тональности (смайлики и восклицательные знаки несут эмоции), при сегментации предложений (точки определяют границы), в анализе чатов (эмодзи заменяют слова).

6. В чем заключается главное различие между процессами лемматизации и стемминга?

Стемминг — это отсечение окончаний для получения псевдоосновы. Лемматизация — это морфологический анализ, направленный на приведение словоформы к её канонической (начальной) форме — лемме. В отличие от стемминга, лемматизация учитывает контекст и флективную структуру языка.

7. Почему для флективных языков (таких как русский) лемматизация эффективнее стемминга?

Лемматизация учитывает контекст и флективную структуру языка, что критически важно для флективных языков, к которым относится русский

язык. Стемминг же работает с псевдоосновами и не учитывает эти особенности.

8. Как алгоритм токенизации определяет границы слов при наличии сокращений (например, «т.д.», «и т.п.»)?

Современные токенизаторы (включая spaCy) используют комбинацию правил и статистических моделей. Они распознают распространённые сокращения как единые токены, не разрывая их по точкам. Для неизвестных сокращений применяются эвристики: точка между заглавными буквами или в конце короткой последовательности часто не считается границей предложения.

9. По какому принципу слова формируют список стоп-слов в библиотеках NLTK или spaCy?

Исключение стоп-слов — это удаление из текста высокочастотных лексических единиц, выполняющих исключительно синтаксическую функцию (предлоги, союзы, частицы). Данные слова обладают низкой информационной ценностью и при векторизации способны искажать результаты работы статистических алгоритмов.

10. Что произойдет с качеством анализа тональности отзывов, если удалить частицу «не» как стоп-слово?

Частица «не» меняет полярность высказывания: «плохо» → «не плохо». При её удалении оба отзыва будут классифицированы как «плохо», что приведёт к систематической ошибке.

11. Какой физический и математический смысл имеет метрика TF (Term Frequency)?

Физический смысл: TF показывает, насколько часто слово встречается в конкретном документе.
Математически: TF = (число вхождений слова в документе) / (общее число слов в документе).

12. Зачем при расчете метрики IDF (Inverse Document Frequency) используется операция логарифмирования?

Логарифм сглаживает резкие перепады значений. Без логарифма отношение N/df может принимать очень большие значения для редких слов, что приводит к числовой нестабильности и преобладанию единичных терминов.

13. Чему будет равен коэффициент TF-IDF для слова, которое встречается абсолютно во всех документах корпуса?

Если слово встречается во всех документах, то отношение (Всего документов / Сколько документов содержат это слово) будет равно 1. Логарифм 1 равен 0. Следовательно, TF-IDF для такого слова будет равен 0.

14. Какая часть метрики TF-IDF отвечает за локальную важность слова, а какая — за его глобальную уникальность?

Метрика рассчитывается как произведение двух независимых сомножителей. TF (Частота слова) отвечает за локальную важность (плотность внутри документа). IDF (Обратная частота документа) отвечает за глобальную уникальность (специфичность термина во всем корпусе).

15. Как длина текста влияет на показатель TF и почему длинные документы могут искусственно занижать вес слов?

TF определяется как отношение количества вхождений слова к общему количеству слов в документе. Чем больше общее количество слов в документе (длиннее текст), тем меньше будет значение TF для одного и того же количества вхождений.

16. Что происходит с размером итоговой матрицы TF-IDF при добавлении в корпус новых уникальных слов?

При добавлении новых уникальных слов в корпус размерность пространства признаков (словаря) увеличивается, что приводит к увеличению размера итоговой матрицы.

17. Что такое «разреженность» матрицы (Sparsity) и почему в итоговой таблице TF-IDF преобладают нули?

Разреженность — это доля нулевых элементов в матрице. В TF-IDF матрице преобладают нули, потому что каждое слово встречается лишь в небольшой части документов корпуса. При большом словаре и относительно малом числе документов большинство ячеек будут нулевыми.

18. Как ошибка определения части речи в библиотеке spaCy может повлиять на итоговый результат лемматизации?

Лемматизация учитывает контекст. Если часть речи определена неверно, каноническая форма (лемма) будет найдена неправильно, что приведет к искажению токена.

19. В чем плюсы и минусы перехода от анализа одиночных слов к анализу словосочетаний (биграмм и триграмм)?

Плюсы: Сохраняется контекст и устойчивые выражения. Снижается полисемия: «свежий лук» и «тугой лук» становятся разными токенами. Повышается качество тематического моделирования.
Минусы: Резко растёт размер словаря, увеличивается разреженность матрицы, требуется больше вычислительных ресурсов, а редкие n-граммы могут не встретиться в обучающей выборке.

20. Что произойдет, если в векторизатор TF-IDF передать новое слово, которого вообще не было в исходном обучающем словаре?

Если слова не было в обучающем словаре, оно не будет учтено при построении матрицы TF-IDF для данного корпуса.

Вывод

В данной лабораторной работе я изучил и освоил основные этапы предобработки текста на русском языке. Я установил библиотеки spaCy и NLTK, сформировал корпус из четырёх документов, очистил текст от шума с помощью регулярных выражений, провёл токенизацию, удаление стоп-слов и лемматизацию через spaCy, а затем объединил эти шаги в единый пайплайн. Вручную рассчитав TF-IDF, я на практике убедился, что редкие слова получают высокий вес, а слова, встречающиеся во всех документах, обнуляются (IDF = 0). В самостоятельных заданиях я изучил список стоп-слов NLTK, добавил кастомные стоп-слова, исследовал масштабируемость корпуса при добавлении нового документа (словарь вырос с 26 до 29 слов, а векторы пересчитались) и модифицировал регулярное выражение для сохранения цифр. С помощью spaCy я проанализировал токенизацию числительных и сделал вывод о том, в каких задачах (анализ чеков, адресов) цифры критически важны, а в каких — являются шумом.